

Unit -V : Strings, Multithreading, JDBC & JavaFX

Strings:

In Java, strings are objects that represents a sequence of characters which may include letters, numbers, symbols, or any other Unicode characters.

Creating Strings:

There are several ways to create strings. We can use following classes to create strings

- a) **String class**
- b) **StringBuffer class**
- c) **StringBuilder class**

So ,In java, a string is an object of class (not an array of characters)

Interface CharSequence

CharSequence is an interface that represents a sequence of characters.

It is implemented by several classes including the classes String, StringBuffer, and StringBuilder.

It has the following four methods.

(i) `charAt(int index)`:

The method returns character value at specified index value.

(ii) `length()`:

This method returns the length of this (invoking) character sequence.

(iii) `subsequence(int startIndex, int endIndex)`:

The method returns a subsequence from start index to end index of this sequence
Throws `IndexOutOfBoundsException`.

(iv) `toString()`:

The method returns a string containing characters of the sequence in the same

```

public class CharSeq
{
    public static void main(String args[])
    {
        CharSequence csq1 = new String("JAVA");
        System.out.println("Second letter in csq1 = " +
            csq1.charAt(1));
        System.out.println("Length of csq1 = " + csq1.length());
        CharSequence csq2 = new StringBuffer("JAVASTRINGS");
        CharSequence csq3 = new StringBuffer("Pavan");
        CharSequence sq = csq2.subSequence(2,6);
        System.out.println("The csq3 = " + csq3);
        System.out.println("The csq2 = " + csq2);
        System.out.println("The sub sequence(2,6) of csq2 sq = " + sq);
    }
}

```

Output:

```

Second letter in csq1 = A
Length of csq1 = 4
The csq3 = Pavan
The csq2 = JAVASTRINGS
The sub sequence(2,6) of csq2 sq = VAST

```

String class:

The String class implements the CharSequence interface and represents a sequence of characters.

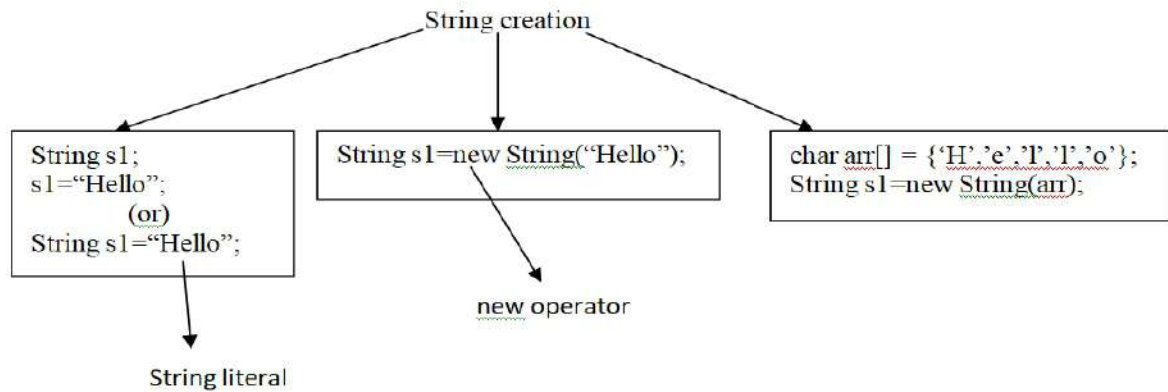
All string literals in Java programs, such as "abc", are implemented as instances of this class.

It is one of the most fundamental classes in Java and is used extensively for handling text data.

Unlike arrays of characters, String objects are immutable, meaning their contents cannot be changed once created.

This immutability provides security and efficiency, as strings can be safely shared among multiple threads.

The following are the different ways to create strings using String class



In Java, characters in a string are numbered with 0-based indexes, meaning the first character has an index of 0, the second character has an index of 1, and so on.

For ex, consider the following string

String s1="Hello";

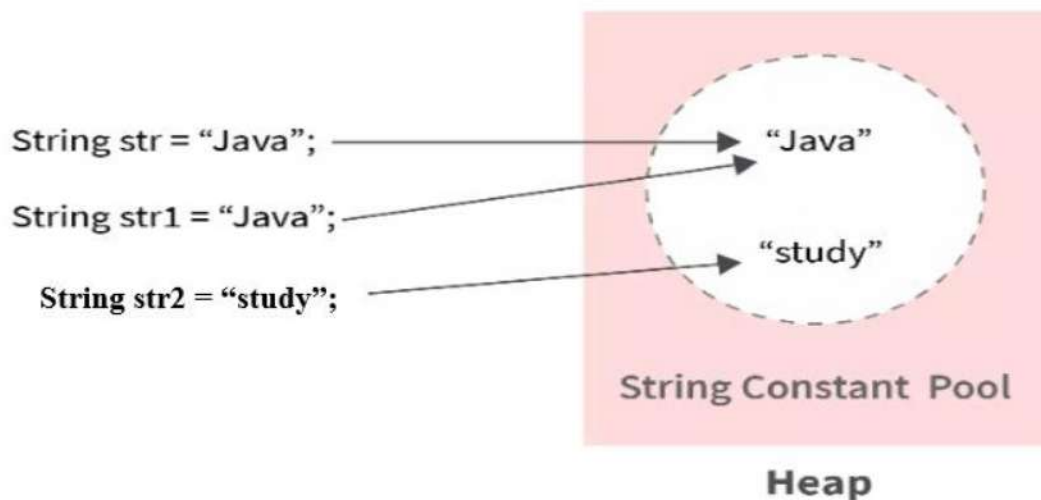
Now the first character 'H' has an index of 0, next character 'e' has an index of 1 and so on. It is illustrated below.

Character →	H	e	l	l	o
Index →	0	1	2	3	4

String Constant Pool:

String Constant Pool is a special storage space in the Heap memory.

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool.



In the above example, Firstly, JVM will not find any string object with the value "Java" in string constant pool that is why it will create a new object. After that it will find the string with the value "java" in the pool, it will not create a new object but will return the reference to the same instance.

String Class Methods:

String class defines several methods that operate on strings to accomplish variety of string manipulation tasks

Methods for Extracting Characters from Strings

The String class in Java provides several methods to extract characters from strings. Here are the most commonly used ones:

1. charAt(int index)

Returns the character at the specified index. Indices start from 0.

```
String str = "Hello";  
char ch = str.charAt(4); //output : 'o'
```

2. substring(int beginIndex)

Returns a new string that starts from the specified index to the end of the string.

Example:

```
String str = "Hello";  
String sub = str.substring(2); // "llo"
```

3. **substring(int beginIndex, int endIndex)**

Returns a new string that starts from beginIndex and ends at endIndex - 1.

```
String str = "Hello";  
String sub = str.substring(1, 4); // "ell"
```

4. **toCharArray()**

Converts the entire string to a new character array.

```
String str = "Hello";  
char[] charArray = str.toCharArray(); // ['H', 'e', 'l', 'l', 'o']
```

5. **getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)**

Copies characters from the specified range of the string into the destination character array.

```
String str = "Hello";  
char[] charArray = new char[5];  
str.getChars(0, 5, charArray, 0); // ['H', 'e', 'l', 'l', 'o']
```

6. **codePointAt(int index)**

Returns the Unicode code point at the specified index.

```
String str = "Hello";  
int codePoint = str.codePointAt(1); // 101 (Unicode code point of 'e')
```

These methods give you various ways to extract specific characters or parts of a string for further manipulation.

Methods for Comparison of Strings:

Java offers several methods for comparing strings. Here's a rundown:

1. equals(Object obj)

Compares the content of two strings. Returns true if the strings have the same sequence of characters.

```
String str1 = "Hello";  
String str2 = "Hello";  
boolean isEqual = str1.equals(str2); // true
```

2. equalsIgnoreCase(String anotherString)

Compares the content of two strings, ignoring case differences.

```
String str1 = "Hello";  
String str2 = "hello";  
boolean isEqualIgnoreCase = str1.equalsIgnoreCase(str2); // true
```

3. compareTo(String anotherString)

Compares two strings lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose s1 and s2 are two String objects. Then

If $s1 == s2$: The method returns 0.

If $s1 > s2$: The method returns a positive value.

If $s1 < s2$: The method returns a negative value.

```
String str1 = "Hello";  
String str2 = "World";  
int comparisonResult = str1.compareTo(str2); // Negative value
```

4. compareToIgnoreCase(String str)

Compares two strings lexicographically, ignoring case differences.

```
String str1 = "Hello";  
String str2 = "hello";  
int comparisonResultIgnoreCase = str1.compareToIgnoreCase(str2); // 0
```

5. contentEquals(CharSequence cs)

Compares the content of this string to the specified character sequence.

```
String str1 = "Hello";  
StringBuffer str2 = new StringBuffer("Hello");  
boolean contentEquals = str1.contentEquals(str2); // true
```

6. regionMatches()

Compares a specific region of one string to a specific region of another string.

```
String str1 = "HelloWorld";  
String str2 = "World";  
boolean regionMatches = str1.regionMatches(5, str2, 0, 5); // true
```

```
class TestStringComparison1  
{  
    public static void main(String[] args)  
    {  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3=new String("Sachin");  
        String s4="SACHIN";  
        System.out.println(s1.equals(s2)); //true  
        System.out.println(s1.equals(s3)); //true  
        System.out.println(s1.equals(s4)); //false  
        System.out.println(s1.equalsIgnoreCase(s4)); //true  
    }  
}
```

Methods for modifying strings:

1. concat(String str)

Concatenates the specified string to the end of this string.

```
String str1 = "Hello, ";  
String str2 = "World!";  
String result = str1.concat(str2); // "Hello, World!"
```

2. replace(char oldChar, char newChar)

Replaces all occurrences of a specified character with a new character.

```
String str = "Hello";  
String result = str.replace('l', 'p'); // "Heppo"
```

3. replaceAll(String regex, String replacement)

Replaces each substring of this string that matches the given regular expression with the specified replacement.

```
String str = "Hello World";  
String result = str.replaceAll("World", "Java"); // "Hello Java"
```

4. replaceFirst(String regex, String replacement)

Replaces the first substring of this string that matches the given regular expression with the specified replacement.

```
String str = "Hello World, World";  
String result = str.replaceFirst("World", "Java"); // "Hello Java, World"
```


5. toLowerCase()

Converts all characters in this string to lowercase.

```
String str = "Hello World";  
String result = str.toLowerCase(); // "hello world"
```

6. toUpperCase()

Converts all characters in this string to uppercase.

```
String str = "Hello World";  
String result = str.toUpperCase(); // "HELLO WORLD"
```

7. trim()

Removes any leading and trailing whitespace from this string.

```
String str = " Hello World ";  
String result = str.trim();
```

8.split(String regex)

Splits this string around matches of the given regular expression.

```
public class MyClass {  
    public static void main(String args[]) {  
        String str = "Hello,World,Java";  
        String[] parts = str.split(",");  
        for(String word:parts)  
            System.out.println(word);  
    }  
}
```

Output:

```
Hello  
World  
Java
```

methods for searching strings:

1. indexOf(String str) and indexOf(char ch)

Returns the index within this string of the first occurrence of the specified substring or character.

```
String str = "Hello, World!";  
int index = str.indexOf("World"); // 7  
int charIndex = str.indexOf('o'); // 4
```

2. **lastIndexOf(String str) and lastIndexOf(char ch)**

Returns the index within this string of the last occurrence of the specified substring or character.

```
String str = "Hello, World!";  
int lastIndex = str.lastIndexOf("o"); // 8  
int lastCharIndex = str.lastIndexOf('l'); // 10
```

3. **contains(CharSequence s)**

Returns true if this string contains the specified sequence of characters.

```
String str = "Hello, World!";  
boolean contains = str.contains("World"); // true
```

4. **startsWith(String prefix)**

Returns true if this string starts with the specified prefix.

```
String str = "Hello, World!";  
boolean startsWith = str.startsWith("Hello"); // true
```

5. **endsWith(String suffix)**

Returns true if this string ends with the specified suffix.

```
String str = "Hello, World!";  
boolean endsWith = str.endsWith("!"); // true
```

StringBuffer Class in Java

The StringBuffer class in Java is part of the java.lang package and is used to create mutable strings. Unlike the String class, which is immutable, StringBuffer allows you to modify the contents of a string without creating a new object.

Key Features

1. **Mutable:** Strings created using StringBuffer can be modified in place without creating new instances.
2. **Thread-Safe:** StringBuffer is synchronized, meaning it is safe to use in a multi-threaded environment.

Constructors

- **StringBuffer():**
Creates an empty StringBuffer instance with an initial capacity of 16.
- **StringBuffer(int capacity):**
Creates an empty StringBuffer instance with the specified initial capacity.
- **StringBuffer(String str):**
Creates a StringBuffer instance with the specified string.
- **StringBuffer(CharSequence cs):**
Creates a StringBuffer instance with the specified character sequence.

Example:

```
StringBuffer buffer1 = new StringBuffer();  
StringBuffer buffer2 = new StringBuffer(50);  
StringBuffer buffer3 = new StringBuffer("Hello");
```

Common Methods

Method Signature	Description
<code>public synchronized StringBuffer append(String str)</code>	It appends (concatenates) the specified string <code>str</code> to the already existing StringBuffer value.
<code>public synchronized StringBuffer insert(int pos, String str)</code>	It inserts the specified string <code>str</code> at the specified position <code>pos</code> .
<code>public synchronized StringBuffer replace(int startIndex, int endIndex, String str)</code>	It replaces the string between the specified <code>startIndex</code> and <code>endIndex</code> with the specified string <code>str</code> .
<code>public synchronized StringBuffer delete(int startIndex, int endIndex)</code>	It deletes a portion of the string from the specified <code>startIndex</code> till <code>endIndex</code> .
<code>public synchronized StringBuffer reverse()</code>	It is used to reverse the given string.

```
//program to illustrate StringBuffer methods
public class Str5 {
    public static void main(String args[]) {
        //creating StringBuffer object using default constructor
        StringBuffer sb = new StringBuffer("This is Text");

        //insert "a Sample" string after "is"
        //index starts with 0 so the index of s is 6
        sb.insert(7, " a Sample");
        System.out.println("after Inserting:"+sb);

        sb.append(" Book");
        System.out.println("after appending:"+sb);

        //replace "Book" with "Message"
        int index=sb.indexOf("Book");
        sb.replace(index,sb.length(),"Message");
        System.out.println("after replacing:"+sb);

        //deleting the substring
        sb.delete(index,sb.length());
        System.out.println("after deleting:"+sb);

        //deleting the character
        sb.deleteCharAt(0);
        System.out.println("after deleting a character:"+sb);
        //reversing the string
        sb.reverse();
    }
}
```

```
System.out.println("after reversing:"+sb);  
  
}  
}
```

Output:

```
after Inserting:This is a Sample Text  
after appending:This is a Sample Text Book  
after replacing:This is a Sample Text Message  
after deleting:This is a Sample Text  
after deleting a character:his is a Sample Text  
after reversing: txeT elpmaS a si sih
```

StringBuilder Class in Java

StringBuilder is part of the java.lang package and is used to create mutable strings.

Unlike String, which is immutable, StringBuilder allows strings to be modified in place without creating new objects. It is similar to StringBuffer but is not synchronized, making it faster and more efficient in single-threaded environments.

Key Features

1. **Mutable:**
Strings created using StringBuilder can be modified directly.
2. **Not Thread- safe:**
StringBuilder is not synchronized, which means it is faster but not safe for use by multiple threads simultaneously.

Constructors

StringBuilder(): Creates an empty StringBuilder with an initial capacity of 16.

StringBuilder(int capacity): Creates an empty StringBuilder with the specified initial capacity.

StringBuilder(String str): Creates a StringBuilder initialized to the contents of the specified string.

```
public class StringBuilderExample {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Hello");  
  
        // Append  
        sb.append(" World");  
        System.out.println(sb.toString()); // "Hello World"  
  
        // Insert  
        sb.insert(5, ",");
```

<pre> System.out.println(sb.toString()); // "Hello, World" // Replace sb.replace(7, 12, "Java"); System.out.println(sb.toString()); // "Hello, Java" // Delete sb.delete(5, 6); System.out.println(sb.toString()); // "Hello Java" // Reverse sb.reverse(); System.out.println(sb.toString()); // "avaJ olleH" } } </pre>
Output: <pre> Hello World Hello, World Hello, Java Hello Java avaJ olleH </pre>

Comparison of String,StringBuffer and StringBuilder:

<i>Property</i>	<i>String</i>	<i>StringBuffer</i>	<i>StringBuilder</i>
Storage area	Constant string pool and heap	Heap	Heap
Object mutability	Not mutable	Mutable	Mutable
Thread safety	Safe	Safe	Not safe
Performance	Fast	Slow	Fast

Multithreading:

- The process of executing multiple threads simultaneously is known as multithreading. It enables maximum utilization of the CPU.
- A thread is a light-weight smallest part of a process that can run concurrently with the other parts(other threads) of the **same process**.
- Threads are **independent** because they all have separate path of execution and share common memory.

Advantages:

- Multithreading saves time as you can perform multiple operations together.
- Multithreading enable maximum utilization of the CPU.
- The threads are independent, so it does not block the user to perform multiple operations at the same time and also, if an exception occurs in a single thread, it does not affect other threads

Application Areas:

- Threads are mainly used in server side programs to serve the needs of multiple clients over the network or internet.
- Threads are also used in the creation of games and animations.

Creating threads:

When we supply a program to the JVM for execution, it creates one thread called main thread and associates the program's main() method. The thread then executes the statements contained in the main() method.

So, In every java program, there is a main thread. Apart from this thread, we can also create our own threads.

A new thread can be created in two ways

- a) creating a class that extends Thread class and override its run() method
- b) creating a class that implements Runnable interface and define the run() method

Thread creation by extending Thread class

The following steps are followed to create a thread

- i) Create a new class that extends Thread class

```
class MyThread extends Thread
{
    .....
    .....
}
```

- ii) Override the run() method, specifying the code that you want to execute in a thread.

```
public void run()
{
    ..... //thread code here
    .....
}
```

- iii) Create an instance of the new class that extends Thread.

- iv) call start() method on that instance to begin execution of the thread

```
//creating thread by extending Thread class
class MyThread extends Thread
{
    public void run()
    {
        for (int I = 0; I < 10; i++)
        {
            System.out.println("Child Thread");
        }
    }
}

public class Demo
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        for (int I = 0; I < 10; i++)
        {
            System.out.println("Main Thread");
        }
    }
}
```


Thread creation by implementing Runnable Interface:

Java does not support multiple inheritance for classes. So if a class has to extend another class, it cannot extend Thread.

But, Java provides an alternative way to create threads where we create a class implementing the Runnable interface.

The following steps are followed to create a thread

- i) Create a class that implements the Runnable interface.
- ii) implement the run() method by placing the code that you want to execute in the thread.
- iii) Create an instance of the newly created class(class that implements Runnable class).
- iv) **Create a Thread object, passing in the Runnable instance.**
(pass a Runnable instance to one of Thread's constructors)
- v) call start() method on the thread instance to begin execution of the thread

```
//creating thread by implementing Runnable interface
class MyThread implements Runnable
{
    public void run()
    {
        for (int I = 0; I < 10; i++)
        {
            System.out.println("Child Thread");
        }
    }
}

public class Demo
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        Thread s= new Thread(t);
        s.start();
        for (int I = 0; I < 10; i++)
        {
            System.out.println("Main Thread");
        }
    }
}
```

Lifecycle of a Thread:

When we start executing a thread based application, there exists 'n' number of threads. During the lifetime of each and every thread, there are many states it can enter. They are.

- ❖ New state
- ❖ Runnable state
- ❖ Running state
- ❖ Blocked or Waiting state
- ❖ Dead or Halted state

New State:

A thread starts its life cycle in the New state. It remains in this state until the program starts the thread using the start() method.

```
Thread thread = new Thread(); // New state
```

Runnable State:

When the start() method is invoked, the thread moves to the Runnable state. The thread is now ready for execution by the thread scheduler.

```
thread.start(); // Runnable state
```

Running State:

The thread scheduler picks the thread from the Runnable pool and moves it to the Running state. The thread's run() method is executed in this state.

```
public void run() {  
    System.out.println("Thread is running...");  
}
```

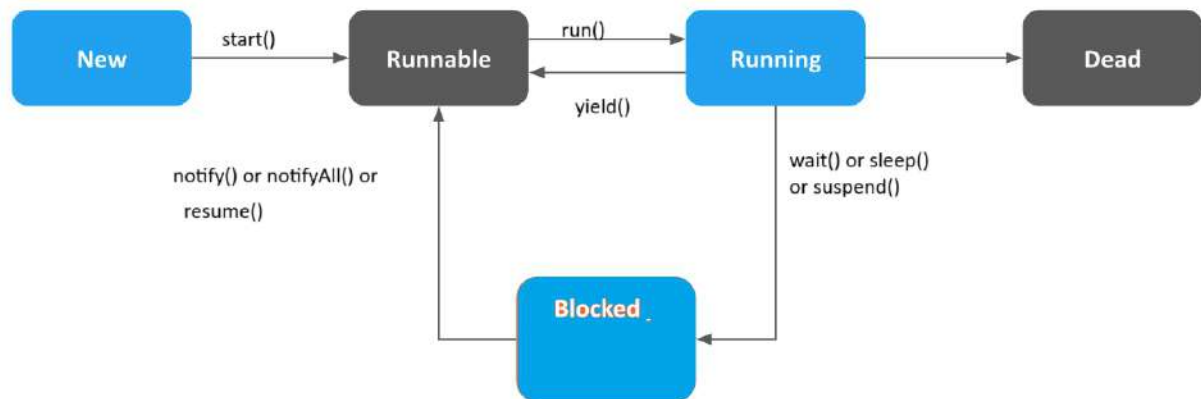
Blocked/Waiting State:

A thread transitions to the Blocked or Waiting state if it needs to wait for a resource or another thread to complete its task. For example, waiting for I/O operations, synchronized blocks, or invoking wait()/join().

```
synchronized (lock) { // Blocked state  
    lock.wait(); // Waiting state  
}
```

Dead: A thread enters this state when its run() method completes or if it's terminated abruptly (like encountering an uncaught exception).

A thread is always in one of these five states, it can move from one state to another state in variety of ways as shown below.



Methods of Thread class:

Some methods of Thread class are given below.

- `getName()`: It is used for Obtaining a thread's name
- `getPriority()`: Obtain a thread's priority
- `isAlive()`: Determine if a thread is still running
- `join()`: Wait for a thread to terminate
- `run()`: Entry point for the thread
- `sleep()`: suspend a thread for a period of time
- `start()`: start a thread by calling its run() method

sleep() method:

`sleep()` is used to temporarily suspend execution of a thread for a given amount of time(sleeping time).

When a thread sleeps, another thread can execute.

After the sleeping time is over, the thread starts its execution from where it has left.

syntax:

public static void sleep(long milliseconds) throws InterruptedException

```
class MyRunnable implements Runnable
{
    public void run()
    {
        go();
    }

    public void go()
    {
        try
        {
            Thread.sleep(1000); //1000 milliseconds or 1 second
        }
        catch(InterruptedException ex)
        {
            ex.printStackTrace();
        }

        doMore();
    }
    public void doMore()
    {
        System.out.println("top of the stack");
    }
}

public class Ex5
{
    public static void main (String[] args)
    {
        Runnable theJob = new MyRunnable();
        Thread t = new Thread(theJob);
        t.start();
        for(int i=0;i<10;i++)
            System.out.println("main method");
    }
}
```

join() method:

Multiple threads run concurrently; one does not wait for the other. But, sometimes it is necessary that a thread should not proceed further until another thread finishes its task. This type of dependency can be achieved using join() method.

join() method allows one thread to wait until another thread completes its execution. In simpler words, it means it waits for the other thread to die.

Syntax:

```
public static void join() throws InterruptedException
```

```
//program to demonstrate join() method
import java.util.Arrays;

class MyThread extends Thread {
    int[] a;

    MyThread(int[] ar) {
        a = ar;
        start();
    }

    public void run() {
        Arrays.sort(a);
        System.out.println("Child completed sorting.");
    }
}

public class Ex6 {
    public static void main(String args[]) throws Exception {
        int b[] = { 2, 6, 4, 0, 1, 5, 3 };
        MyThread t = new MyThread(b);
        t.join(); // wait for t to terminate
        System.out.println("Main printing array elements are :");
        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
    }
}

//main thread
//new thread---job of new thread is sorting the array
//job of main thread is printing the sorted array

//program where I need to create two threads.
//In this program, I need to sort an array and I need to print the sorted array elements.

//single program, I have two subtasks--sorting an array and printing the sorted array
elements--
//Thread1---job is sorting an array---newly created thread
//Thread2---job is printing the sorted array elements--main thread
```

//after Thread1 finishes the task, then only Thread2 can finish its task

//Thread2 need to wait until thread1 finishes its task. This can be done using join() method.

Thread Synchronization

when two or more threads need access to a shared resource that can be used by only one thread at a time, it is necessary to prevent one thread from accessing an resource that is currently in use by another.

The mechanism by which access to an object by multiple threads is controlled is called synchronization.

When two or more threads access the same object, the state of the object is not guaranteed to be correct all the time if special care is not taken. It is illustrated in the following figure.

<pre>class Counter { int val=0; void inc() { val++; } void dcr() { val--; } }</pre>	If these two methods on a Counter object are invoked same number of times in a multithreaded environment , then the state of the object may not be correct.
---	--

- synchronization is supported by the keyword synchronized
- There are two ways to synchronize access to an object.
 - synchronized Methods
 - synchronized Blocks

Thread Synchronization –synchronized method

- modify one or more of object methods with the synchronized keyword so that **only one thread at a time** can access it.
- A synchronized method has the following general form:

<pre>synchronized type methodName(arg-list) { // synchronized method body }</pre>	<pre>class Counter { int val = 0; synchronized void inc() { val++; } synchronized void dcr() { val--; } }</pre>
---	---

When a synchronized method is called on an object, that object is locked by the calling thread.

No other thread can execute a synchronized method on the object until the thread releases the lock, either by returning from the method or by calling the wait() method.

Thread Synchronization –synchronized block

The second way to synchronize access to an object is to use a synchronized block.

A synchronized block has the following general form:

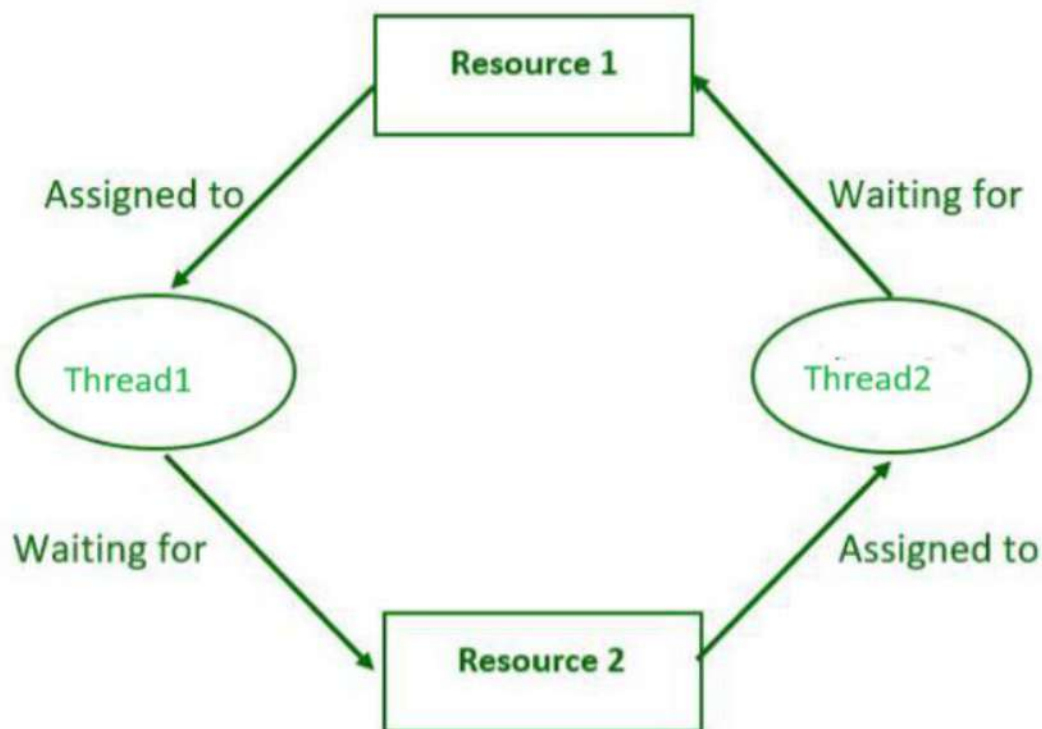
<pre>synchronized(objref) { //synchronized statements }</pre>	<pre>class Counter { int val = 0; Object ob = new Object(); void inc() { synchronized (ob) { val++; } } void dcr() { synchronized(ob) { val--; } } }</pre>
---	--

Once a synchronized block has been entered, objref is locked and no other thread can acquire that lock until the block ends.

So, a synchronized block ensures that a call to a method on objref proceeds only after the current thread has acquired objref's lock.

Deadlocks:

- Deadlock is a situation where two or more threads wait for each other indefinitely to release resources
- A deadlock may occur when multiple threads need the same locks on resources but obtain them in different order



```
class MyThread extends Thread
{
    String r1, r2;
    int id;
    MyThread(int i, String s1, String s2)
    {
        id = i;
    }
}
```



```

        r1 = s1;
        r2 = s2;
        start();
    }
    public void run()
    {
        synchronized(r1)
        {
            System.out.println("Thread "+id+" obtained a lock on
"+r1);

            try
            {
                Thread.sleep(1000);
            }
            catch(Exception e){ }

            System.out.println("Thread "+id+" is waiting to obtain a
lock on "+r2);

            synchronized(r2)
            {
                System.out.println("Thread "+id+" obtained a lock
on "+r2);
            }
        }
    }
}
public class DeadLockDemo
{
    public static void main(String args[]) throws Exception
    {

        String r1 = new String("R1");
        String r2 = new String("R2");
        MyThread t1 = new MyThread(1, r1, r2);
        MyThread t2 = new MyThread(2, r2, r1);
    }
}

```

//observe the program never terminates since deadlock occurs //press ctrl+z

```
// to prevent deadlock, make sure that all locks are //always taken in the same
order by
//any thread
//modify the two lines as follows
//MyThread t1 = new MyThread(1, r1, r2);
//MyThread t2 = new MyThread(2, r1, r2);
/*
Another way to prevent deadlock requires threads to acquire all the locks that
are needed during execution before proceeding.
synchronized(MyThread.class) {
    lines 14 to 27 run() method code
*/
*/
```

Interthread Communication:

Interthread communication allows threads to collaborate by notifying each other about the state changes and resource availability, ensuring smooth execution and resource management.

Threads communicate using the following methods

- wait()
- notify()
- notifyAll()

These methods can be called only from within a synchronized method or synchronized block of code otherwise, an exception named `IllegalMonitorStateException` is thrown.

All these methods are declared as final in Object Class.

i) wait() method:

wait() method in Java notifies the current thread to give up the monitor (lock) and to go into sleep state until another thread wakes it up by calling notify() or notifyAll() method. This method throws `InterruptedException`.

ii) notify() method:

The notify() method wakes up a single thread that called wait() method on the same object. If more than one thread is waiting, this method will awake one of them.

iii) notifyAll() method:

The notifyAll() method is used to wake up all threads that called wait() method on the same object. The thread having the highest priority will run first.

Producer consumer problem:

The producer- consumer problem is a classic example of a multi-threading challenge in computing.

It involves two types of threads, producers and consumers, which share a common buffer.

- **Producers:** This thread produces data and places it into the shared buffer.
- **Consumers:** This thread consumes data from the buffer.

The challenge lies in ensuring that producers don't overwrite data before consumers have processed it, and vice versa. This prevents data loss and corruption.

```
public class PrimeProducerConsumer extends Thread {
    private static final int BUFFER_SIZE = 100;
    private static final int MAX_NUMBER = 100; // Arbitrary upper limit for number
    generation
    private static int turn = 0;
    private int[] buffer;
    private int nextNumber = 2;

    public PrimeProducerConsumer(int[] buffer) {
        this.buffer = buffer;
        start();
    }

    // Efficient prime number check
    private boolean isPrime(int number) {
        if (number <= 1) return false;
        if (number == 2) return true;
        if (number % 2 == 0) return false;
        for (int i = 3; i * i <= number; i += 2) {
```

```

        if (number % i == 0) return false;
    }
    return true;
}

// Produce next prime number
private int nextPrime() {
    while (nextNumber <= MAX_NUMBER) {
        if (isPrime(nextNumber)) {
            return nextNumber++;
        }
        nextNumber++;
    }
    return -1; // Signal end of prime number generation
}

public void run() {
    while (true) {
        synchronized (buffer) {
            while (turn != 0) {
                try {
                    buffer.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            int prime = nextPrime();
            if (prime == -1) break; // End of prime numbers
            buffer[0] = prime;
            turn = 1;
            buffer.notify();
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    int[] buffer = new int[1]; // Single-element buffer to store prime numbers
    PrimeProducerConsumer producer = new PrimeProducerConsumer(buffer);

    while (true) {
        synchronized (buffer) {
            while (turn != 1) {
                buffer.wait();
            }
            int prime = buffer[0];
            if (prime == -1) break; // End of prime numbers
            System.out.print(prime + " ");
            turn = 0;
            buffer.notify();
        }
    }
}

```

```
}  
}  
}  
}
```

Output:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Introduction to JDBC:

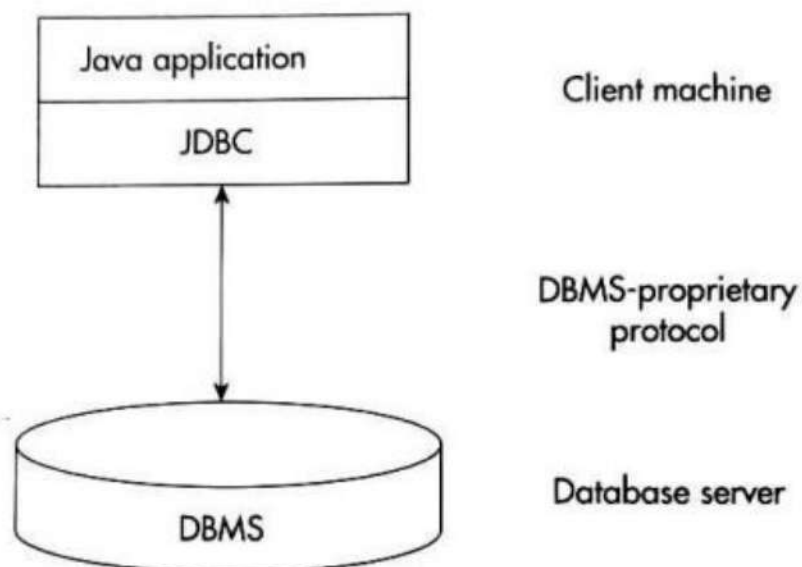
- **Java Database Connectivity (JDBC)** is an API (Application Programming Interface) in Java that allows Java applications to interact with a wide range of databases.
- JDBC is part of the Java Standard Edition platform, which provides a set of classes and interfaces for performing database operations such as querying, updating, and managing database data.
- It acts as a bridge between Java programs and databases, enabling developers to execute SQL statements and retrieve data from databases using Java code.

JDBC Architecture

JDBC architecture defines how Java applications communicate with a database using a set of standard interfaces and classes. There are two main types of JDBC architecture:

1. **Two-Tier Architecture (Direct Connection):**

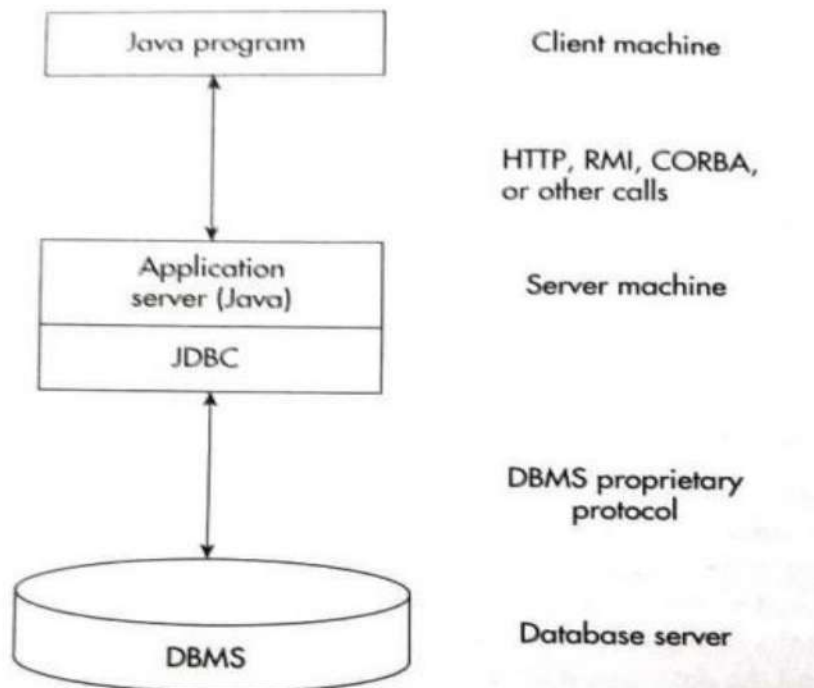
- In a two-tier architecture, a Java application directly connects to the data source using the JDBC API.
- The application communicates with the database server using the database's specific JDBC driver.



- **Suitable for standalone applications where the client and database reside on the same machine or on different machines connected over a network.**

2. Three-Tier Architecture (Client-Server Model):

- In a three-tier architecture, the Java application acts as the client, and it communicates with an application server or a middleware component.
- The application server then interacts with the database.



- This architecture provides better scalability and is typically used in web applications where multiple clients connect to a central server, which in turn connects to the database.

Basic Steps to Use JDBC:

The following steps are used to work with JDBC.

1. **Load or Register the JDBC Driver**
2. **Establish a Connection to the Database**
3. **Prepare SQL Statement**
4. **Execute the Query**
5. **Retrieve the ResultSet**
6. **Close the Connection**

1. Load the JDBC Driver

This step loads the JDBC driver into memory, enabling your application to interact with the database.

The `Class.forName("com.mysql.cj.jdbc.Driver")` method dynamically loads the driver class during runtime.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

This tells the JVM to load the `com.mysql.cj.jdbc.Driver` class, which registers itself with the `DriverManager`.

2. Establish a Connection to the database

To connect to your database, you need to provide a connection URL, along with database username and password.

The `DriverManager.getConnection()` method establishes a connection to the specified database.

Ex:

```
String jdbcUrl = "jdbc:mysql://localhost:3306/yourDatabase";  
String user = "yourUsername";  
String password = "yourPassword";  
Connection con = DriverManager.getConnection(jdbcUrl, user, password);
```

This establishes a connection object that can be used to execute SQL queries.

3. Prepare SQL Statement

A statement object is used to execute SQL queries against the database. You can create a `Statement`, `PreparedStatement`, or `CallableStatement` based on requirements.

- `Statement`:

For simple SQL queries without parameters.

- `PreparedStatement`:

For queries with parameters, offering better performance and security.

- `CallableStatement`:

For calling stored procedures in the database.

Ex:

```
Statement st = con.createStatement();
```

4. Execute the Query

Using the created statement object, you can execute SQL queries. The

`executeQuery()` method is used for SELECT queries and

`executeUpdate()` is used for INSERT, UPDATE, and DELETE queries.

Ex:

```
// For SELECT query
ResultSet rs = st.executeQuery("SELECT * FROM employees");

// For INSERT, UPDATE, DELETE queries
int rowsAffected = st.executeUpdate("INSERT INTO employees (name, salary) VALUES ('Pavan', 5000)");
```

5. Retrieve the ResultSet

The ResultSet object contains the result of the SELECT query. You can iterate through the ResultSet to access the retrieved data.

Ex:

```
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    double salary = rs.getDouble("salary");
    System.out.println("ID: " + id + ", Name: " + name + ", Salary: " + salary);
}
```

6. Close the Connection

Closing the ResultSet, Statement, and Connection objects is crucial to free up database resources and prevent memory leaks.

Ex:

```
rs.close();
st.close();
con.close();
```

```
// java program that connects to a database and create a table using JDBC

package com.malineni.JDBCDEMO1;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class App {
    public static void main(String[] args) {
        // Database URL, user name, and password
        String url = "jdbc:mysql://localhost:3306/malineni"; // Replace malineni with your
        database name
        String username = "root"; // Replace with your database username
        String password = "root"; // Replace with your database password

        try {
            // Step 1: Load or register the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish a connection to the database
            Connection con = DriverManager.getConnection(url, username, password);

            // Step 3: Create a Statement object
            Statement st = con.createStatement();

            // Step 4: Execute the SQL statement
            // SQL statement to create a new table
            String query= "CREATE TABLE users (userid int primary key, username
                           varchar(20), password varchar(20))";

            st.executeUpdate(query);
            System.out.println("Table users created successfully!");

            // Step 5: Close the resources
            st.close();
            con.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Table users created successfully!

```
// java program to connect to a database using JDBC and insert values into it and then select.
```

```
package com.malineni.JDBCDemo2;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class App
{
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/malineni";
        String user = "root";
        String password = "root";

        try {
            // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Establish the connection
            Connection con = DriverManager.getConnection(url, user, password);

            // Create a statement
            Statement st = con.createStatement();

            // Execute a query
            String query = "insert into users (userid, username, password) values (3, 'Raju', 'khanna')";

            int rowsAffected = st.executeUpdate(query);
            System.out.println("One record is inserted into the users table");

            System.out.println("Fetching the records");
            String query2 = "select * from users";

            // Process the result set
            ResultSet rs = st.executeQuery(query2);

            while (rs.next()) {
                int id = rs.getInt("userid");
                String name = rs.getString("username");
                System.out.println("ID: " + id + ", Name: " + name);
            }

            // Close the resources
```

```

        rs.close();
        st.close();
        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Output:

One record is inserted into the users table
 Fetching the records
 ID: 1, Name: pavan
 ID: 2, Name: gowtham
 ID: 3, Name: Raju

//Write a java program to connect to a database using JDBC and delete values from it

```

package com.malineni.JDBCDemo3;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class App {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/malineni";
        String user = "root";
        String password = "root";

        try {
            // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish the connection
            Connection con = DriverManager.getConnection(url, user, password);
            // Create a statement
            Statement st = con.createStatement();
            // SQL query to delete a record
            String query = "DELETE FROM users WHERE userid = 3";
            // Execute the query
            int rowsAffected = st.executeUpdate(query);

            if (rowsAffected > 0) {

```

```
        System.out.println("Record deleted successfully.");
    } else {
        System.out.println("No record found with the specified userid.");
    }

    // Close the resources
    st.close();
    con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
```

Output:

Record deleted successfully.